4.5 Встраивание типов и композиция в Go

В Go отсутствует традиционное **наследование**, характерное для языков вроде Java или C++. Вместо этого язык предлагает два механизма для повторного использования кода и построения сложных типов: **встраивание** (embedding) и **композицию**.

Встраивание типов — это процесс включения одного типа (структуры или интерфейса) в другой без указания имени поля. Позволяет продвигать поля и методы встроенного типа к внешнему типу, упрощая доступ к ним.

- **Для структур**: поля и методы встроенной структуры становятся доступными напрямую.
- **Для интерфейсов**: встраивание объединяет требования нескольких интерфейсов в один составной интерфейс.

Пример встраивания структур

```
package main

type Person struct {
  Name string
}
func (p Person) Greet() {
  println("Hello, I am " + p.Name)
}
type Employee struct {
  Person // Встраивание типа Person  Salary int
}
func main() {
  emp := Employee{Person{"Alice"}, 50000}
  println(emp.Name, emp.Person.Name, emp.Salary)
  emp.Greet()// Метод Greet продвигается от Person
}
```

Приоритет методов при встраивании

```
package main

type Person struct {
  Name string
}

func (p Person) Speak() {
  println("Person speaking")
}

type Employee struct {
  Person
}

func (e Employee) Speak() {
  println("Employee speaking")
}

func main() {
   e := Employee{Person{"Alice"}}
   e.Speak()       // Вызовет метод Employee
   e.Person.Speak() // Вызовет метод Person
}
```

Встраивание интерфейсов

```
package main

type Reader interface {
  Read()
}

type Writer interface {
 Write()
}

type ReadWriter interface {
   Reader // Встраивание интерфейса Reader
   Writer // Встраивание интерфейса Writer
}
type File struct{} func (f File) Read() {
   println("Reading")
}

func (f File) Write() {
println("Writing")
}

func main() {
   var rw ReadWriter = File{}
   rw.Read() rw.Write()
}
```

Композиция

Композиция в Go — это способ построения сложных типов путем включения других типов как именованных полей в структурах. В отличие от встраивания, поля и методы не продвигаются автоматически, и доступ возможен только через имя поля.

Пример композиции

```
package main

type Address struct { Street
   string
  City  string
}

type Person struct { Name
   string
   Address Address // Композиция: Address как именованное поле
}

func main() {
   p := Person{Name: "Alice", Address: Address{Street: "123 St",
   City: "City"}}
   println(p.Address.Street) // Доступ через имя поля
   // println(p.Street)    // Ошибка: p.Street не существует
}
```

Проблема продвижения методов при встраивании

При встраивании нескольких типов с одинаковыми именами методов или полей возникает неоднозначность.

```
package main

import "fmt"

type Writer struct{}

func (w Writer) Write() {
  fmt.Println("Write method of Writer")
}

type Logger struct{}

func (l Logger) Write() {
  fmt.Println("Write method of Logger")
}

type MyStruct struct { Writer
  Logger
}

func main() {
  m := MyStruct{}
  m.Write() // Ошибка: ambiguous selector m.Write
}
```


MERION

Решение проблемы неоднозначности

1. Использование композиции

```
package main

import "fmt"

type Writer struct{}

func (w Writer) Write() {
fmt.Println("Write method of Writer")
}

type Logger struct{}

func (l Logger) Write() {
fmt.Println("Write method of Logger")
}

type MyStruct struct {
writer Writer
logger Logger
}

func main() {
   m := MyStruct{}
   m.writer.Write() // Вызовет Write от Writer
   m.logger.Write() // Вызовет Write от Logger
}
```

2. Явное определение метода

```
package main

import "fmt"

type Writer struct{}

func (w Writer) Write() {
  fmt.Println("Write method of Writer")
}

type Logger struct{}

func (l Logger) Write() {
   fmt.Println("Write method of Logger")
}

type MyStruct struct { Writer
   Logger
}

func (m MyStruct) Write() {
   m.Writer.Write() // Явно указываем, какой метод использовать
}

func main() {
   m := MyStruct{} m.Write()
}
```

Ключевая разница между встраиванием и композицией

Аспект	Встраивание (без имени)	Композиция (с именем поля)
Синтаксис	Person (без имени)	Address Address (с именем)
Продвижение	Автоматическое для полей и методов	Нет продвижения, доступ через имя
Доступ	emp.Name	p.Address.Street
Неоднозначность	Возможны конфликты	Конфликтов нет
Использование	Для отношений "является" (is-a)	Для отношений "содержит" (has-a)

Преимущества встраивания

1. **Повторное использование кода**: Поля и методы встроенного типа доступны без дублирования.
2. **Упрощение кода**: Автоматическое продвижение делает код более лаконичным.
3. **Установление отношений**: Встраивание явно показывает связь "является" между типами.
4. **Гибкость**: Позволяет комбинировать несколько типов, создавая модульные структуры.